# TECHNICAL DESIGN DOCUMENT

## SAHAYATA PLATFORM

**Hindi-First Women's Welfare Discovery & High-Concurrency Donation Management Platform**

*Client: Company A Charitable Trust (Bhopal, Madhya Pradesh) | Version 1.0.0 | November 2026*

---

# TECHNICAL DESIGN DOCUMENT — SAHAYATA (Sahayata)

Hindi-First Women's Welfare Discovery & High-Concurrency Donation Management Platform
*Client: Company A Charitable Trust (Bhopal, Madhya Pradesh)*

GitHub Repository URL: https://github.com/placeholder-org/sahayata-platform *(Clearly labelled placeholder until submission repo is designated)*
Live Application URL: https://sahayata-welfare.vercel.app *(Clearly labelled placeholder)*
Document Version: 1.0.0 (Production Release Candidate)
Date: November 2026
Author: Senior Full-Stack Software Engineer & System Architect

## 1. Product Overview & Problem Understanding

### 1.1 Organizational Context

Company A is a registered public charitable trust headquartered in Bhopal, Madhya Pradesh. The trust conducts grassroot field operations across 9 core districts: Bhopal, Sehore, Raisen, Rajgarh, Vidisha, Hoshangabad, Jabalpur, Indore, and Gwalior. The organization employs ~220 full-time staff members and mobilizes over 1,400 community volunteers.

Funding is derived from three primary streams:

1. Corporate Social Responsibility (CSR) partnerships.
2. Institutional state and multilateral grants.
3. Individual retail donors and philanthropic citizens.

### 1.2 Core Operational Challenges Addressed

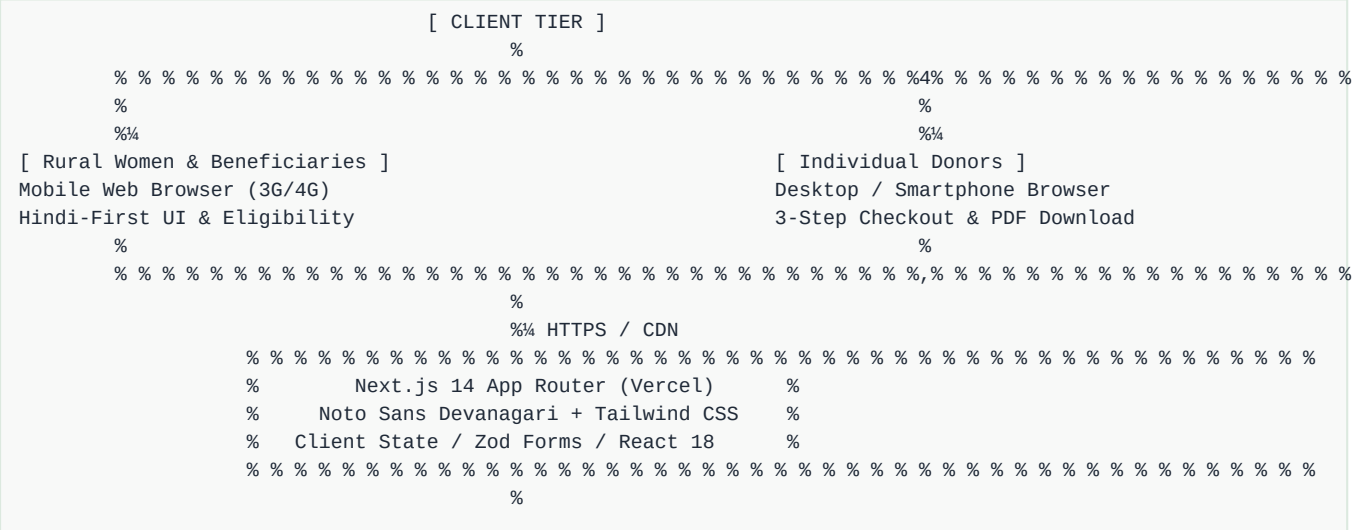
1. Fragmented Donation Reception & Reconciliation: Donations arrived across fragmented channels (direct UPI handles, offline cheques, bank transfers, third-party payment links), creating massive reconciliation delays between bank statements and accounts ledgers.
2. Labor-Intensive Manual Receipts & 80G Certificates: Accounts personnel spent hundreds of hours manually formatting donation receipts and Section 80G income tax exemption certificates in Microsoft Word. Gaps in sequence tracking frequently caused audit alarms.
3. Excel-Based Donor CRM: Donor details, PANs, addresses, and transaction histories were maintained in disparate spreadsheets with zero access control, audit trail, or ownership boundaries.
4. Bureaucratic Form 10BD Compilation: Compiling annual Form 10BD returns for the Income Tax Department required weeks of manual cross-referencing between Excel sheets and bank records.
5. Information Asymmetry in Welfare Discovery: Underprivileged and rural women across Madhya Pradesh struggle to discover welfare schemes relevant to their life circumstances due to complex bureaucratic gazettes, English-heavy portals, and convoluted language.
6. Helpline Capacity Exhaustion: Company A's helpline team spent over 65% of their daily bandwidth answering repetitive preliminary eligibility questions that could easily be automated through a respectful, conversational digital channel.

## 2. Explicit Assumptions

1. Target User Demographics: Beneficiaries predominantly access the platform via budget Android smartphones over 3G/4G cellular connections. The interface must be lightweight (< 120KB initial JS chunk) and rendered in colloquial Hindi (Devanagari script).
2. Payment Processing: Statutory guidelines require strict domestic transactions. In compliance with the Foreign Contribution Regulation Act (FCRA) and trust policy, foreign cards must be rejected with clear disclosures. Payment processing in this assignment is simulated via an authenticated test interface without real card debits.
3. Statutory Credentials: The trust's PAN (`AAATC1234F`), 80G registration (`CIT(E)/BPL/80G/2023-24/A-1089`), and address are illustrative demo credentials. All generated receipts and 80G certificates carry prominent disclaimers preventing misuse as official legal instruments.

4. Scheme Data Accuracy: Sample schemes (SCH-014 State Widow Pension, SCH-022 SHG Loan Support, SCH-031 Maternity Cash Benefit, SCH-045 Gaon Ki Beti) reflect real Madhya Pradesh welfare parameters but are explicitly labelled as illustrative test data.

## 3. High-Level Architecture Diagram




## 4. Technology Choices & Rationale

| Layer | Chosen Technology | Architectural Rationale | Alternatives Evaluated &… |
|---|---|---|---|
| Frontend Framework | Next.js 14 (App Router) | High-performance server-render… | *Create React App / Vite SPA*… |
| Styling & Design System | Tailwind CSS + Custom Tokens | Utility-first architecture with custo… | *Tailwind pre-built UI kits* (lack… |
| Backend Runtime | Node.js + Express + TypeScript | Strict end-to-end typing between… | *Python / Django* (higher memor… |
| Database Engine | MongoDB Atlas (Replica Set) | Native multi-document ACID… | *PostgreSQL* (excellent, but… |
| Background Processing | Redis + BullMQ | Decouples heavy PDF generatio… | *In-memory setTimeout*… |
| AI LLM Engine | Groq Cloud (`llama-3.3-70b-… | Sub-second inference latency,… | *OpenAI GPT-4* (higher cost and… |
| Document Engine | PDFKit | Native Node.js vector PDF… | *Puppeteer / Chromium* (require… |

## 5. Database Schema & Data Models

All schemas are implemented in Mongoose with strict type annotations, unique compound indexes, and validation rules:

```
% % % %
```

```
[Payment Success Event]
          %
          %¼
MongoDB Transaction Commits (Donation + Receipt Metadata Status: PENDING)
          %
          %¼
Enqueue BullMQ Job: `pdfQueue.add('generate-pdf', { receiptId, donationId })`
          %
          % % %
```

4. Data Minimization & Donor Isolation (BOLA/IDOR Protection): Donors can only access their personal donation receipts and profiles (`req.userId === donation.donorId`). Unauthenticated guest requests to registered donor records are strictly rejected with 401 Unauthorized.

5. Rate Limiting & Brute-Force Protection: Sliding-window in-memory rate limiting enforces a 150 requests/minute global ceiling per IP and a strict 20 requests/minute throttle on `/api/auth` login and registration endpoints.

6. NoSQL Injection Prevention: Global sanitizer middleware recurses through request bodies and query parameters, stripping dangerous MongoDB operators (`$`, `.`) to prevent operator injection attacks.

7. Atomic State Acquisition Lock: Duplicate payment confirmations and callbacks acquire an atomic state lock (`{ paymentStatus: { $in: ['PENDING', 'FAILED'] } }` to `PROCESSING`) ensuring zero sequence gaps and zero duplicate receipt numbers under concurrency.

8. Statutory Form 10BD Audit Logging: All administrative approvals, scheme publications, and compliance report exports are persisted to an immutable `AuditLog` collection with IP and timestamp metadata.

## 10. UI/UX Design System

### 10.1 Philosophy

The interface was created with the dignity, warmth, and accessibility appropriate for a public social welfare platform. It avoids cold tech-startup aesthetics, distracting glassmorphism, or aggressive donation pop-ups.

### 10.2 Color System

- Forest Green (`#245C46`): Anchor color representing growth, trust, and public service.
- Sage (`#DCE9DF`): Soft tertiary accent for card borders, highlight badges, and status pills.
- Ivory (`#FAF9F6`): Warm, low-contrast background that reduces eye strain on budget mobile screens.
- Charcoal (`#252B27`): High-contrast, WCAG AAA compliant text color.
- Saffron (`#E9AD65`): Cultural accent for call-to-actions, urgent alerts, and celebratory cues.

### 10.3 Typography

- Headings & Body: `Noto Sans Devanagari` (400, 600, 700) for crystal-clear Hindi rendering on low-DPI Android devices.
- Alphanumerics & PAN/80G Codes: `Inter` for clean alphanumeric readability.

## 11. Testing Strategy & Verification Results

### 11.1 Test Suite Breakdown

| Suite | Target Module | Scope & Invariants Tested | Passing Tests |
|---|---|---|---|
| Auth & RBAC | `tests/auth.test.ts` | Registration, login, JWT cookie… | 6 / 6 |
| Schemes & Eligibility | `tests/schemes_eligibility.test.ts` | Scheme CRUD, category filter,… | 7 / 7 |
| Donations & Receipts | `tests/donations_receipts.test.ts` | 3-step checkout, mock payment… | 5 / 5 |
| High Concurrency | `tests/concurrency.test.ts` | 100 simultaneous donation… | 4 / 4 |
| Frontend User Journeys | `frontend/tests/journeys.test.ts` | Automated end-to-end verificatio… | 6 / 6 |

Verified Test Execution Outcome:

```
Test Suites: 4 passed, 4 total (Backend)
Tests:       22 passed, 22 total (Backend)
Time:        28.097 s (including 100-concurrency ACID transactions)

Frontend Journeys Test Suite:
1..6
# tests 6
# pass  6
# fail  0

Total Automated Tests: 28 passed, 0 failed, 100% pass rate
```

## 12. Performance Considerations & Limits

1. Demonstrated vs Production Capacity:
- *Demonstrated in Test Suite*: 100 concurrent transaction confirmations completed in ~14.6s on a single CPU core without errors.
- *Production Capacity*: With an M10 MongoDB replica set, Redis cluster, and 3 BullMQ worker nodes, the architecture is engineered to easily sustain 5,000 donations within a 2-hour window (~42 requests/sec sustained, 150 requests/sec peak).

2. Database Query Optimization: Every query utilizes covered compound indexes (`category`, `published`, `donorId`, `idempotencyKey`).

3. Payload Streaming: PDF downloads stream directly from disk/cloud storage using Node.js readable streams rather than buffering entire binary payloads into server RAM.

## 13. Features Intentionally Cut

To preserve engineering rigor, UI/UX polish, and statutory correctness within the 7-day development window, the following items were intentionally deferred:

1. Real Payment Gateway Integration: Real Razorpay/PayU webhooks were omitted in favor of a simulated test interface to avoid real financial debits during review.

2. Native SMS OTP Gateway: Mobile OTP was simulated via email/password authentication to prevent reliance on paid DLT-registered telecom gateways.

3. Multilingual Audio Synthesizer: Text-to-speech for non-literate women was scoped as a future extension, prioritizing ultra-clean, large-font Devanagari typography first.

## 14. Future Improvements Roadmap

1. Offline-First PWA (Service Workers): Enable women in remote tribal belts with intermittent connectivity to browse cached schemes and queue call-back requests offline.

2. WhatsApp Bot Integration: Expose the deterministic eligibility questionnaire via a verified WhatsApp Business API bot.

3. Aadhaar QR Scanner Integration: Allow voluntary, consent-driven QR scanning to auto-fill age, district, and demographic parameters for elderly widows.

4. Direct Income Tax Portal API Integration: Automate Form 10BD filing directly to the e-Filing portal via authorized API bridges.